
Distractor Filtering Through Open Knowledge Graph Concept Proximity for Dense RAG Systems

June 11, 2026

Gianluca Como (1605533)

Abstract

Dense retrieval can surface *distractors* — passages close to the query but answer-free — that degrade RAG accuracy, as Cuconasu et al. [3] show. We test whether *pure graph topology* on Wikidata, with no relation semantics, can filter them. From a local HDT dump we build a 661.5M-edge entity graph, score query–document pairs by topological reachability (connected $\times$ purity), and fuse this with the dense score across a grid of hop-distance, hub-threshold and mixing weight α , evaluating on 1000 NQ-open queries with an LLM judge. The result is a clean *negative*: the KG signal never beats the dense baseline (0.364) beyond noise (best 0.372) and actively degrades accuracy as its weight grows (down to 0.290); 13/90 conditions are significant under McNemar+Bonferroni, all worse. We explain why and document the engineering and the departures from Cuconasu et al. [3].

1. Introduction

Cuconasu et al. [3] show that high-scoring retrieved passages are often *distractors* — semantically similar to the query yet answer-free — and degrade the reader. We test one idea against this: filter distractors by *graph proximity* on an open knowledge graph, reranking retrieved passages by how close their entities are, on Wikidata, to the query entities. We deliberately restrict ourselves to pure topology — bare ≤ 3 -hop reachability, no relation semantics — to isolate what proximity *alone* contributes. The answer is negative: topology alone does not filter distractors. The report’s main contribution is this controlled test and the mechanistic account of *why* it fails; a by-product is a method to decide ≤ 3 -hop reachability over the entire

Wikidata graph without materialising hub neighbourhoods. Code {4}.

2. Related work

We adopt the corpus, retriever and reader of Cuconasu et al. [3], who characterise the distractor problem in RAG. Retrieval uses Contriever over the gold-augmented Wikipedia corpus of Cuconasu et al. [3]; entity linking uses the ReFinED linker of Ayoola et al. [1], the best-performing system in the independent evaluation of Bast et al. [2]. Knowledge-graph approaches to RAG use the graph’s relations and semantics; we instead use only its topology.

3. Method

3.1. Baseline: dense RAG

We reproduce the dense-retrieval pipeline of Cuconasu et al. [3]. The corpus is its gold-augmented Dec-2018 Wikipedia release {1}; we re-segment it on sentence boundaries and pad each passage to exactly 100 words — the raw DPR passages have ragged, subject-less tails that hurt entity linking and the reader, and since Contriever mean-pools, fixed length is only a consistency choice. Passages are Contriever-encoded and indexed with FAISS (IndexFlatIP, ~ 42 M). From NQ-open {2} we keep short (≤ 5 -token) answers — matching the extractive setup of Cuconasu et al. [3] — whose question and *all* answer variants link to Wikidata entities (ReFinED); we evaluate on a curated 1000-query subset. The top-5 passages feed Llama-2-7B (base) under a one-shot extraction prompt, and correctness is decided by an *LLM judge* (Qwen2.5-7B-Instruct, a different model to avoid self-preference) that checks whether the response contains an NQ gold answer — replacing the brittle substring exact-match of Cuconasu et al. [3] (prompt and judge in the Appendix).

3.2. Contribution: KG concept-proximity reranking

We rerank the top-100 by graph proximity on Wikidata. We query a pre-built HDT dump {3} (Jan 2022, ~ 166 GB) via pyHDT — the public SPARQL endpoint timed out even

Email: Gianluca Como <comogianluca@gmail.com>.

on a single hub count (see Appendix) — and export every direct entity-to-entity statement (Wikidata’s “truthy” wdt : predicates), 661.5M edges. For a pair (q, d) we test $\text{dist}(q, d) \leq 3$ *meet-in-the-middle*: distance 1 a set lookup, distance 2 an intersection of 1-hop neighbourhoods N_1 , distance 3 an indexed edge-probe between $N_1(q)$ and $N_1(d)$ — never materialising a hub’s neighbour list. A degree threshold t drops high-degree bridges. The KG score is $s_{\text{kg}} = \text{cr} \cdot \text{pr}$, where the *connected ratio* cr is the fraction of query entities reaching ≥ 1 document entity within k hops and the *purity ratio* pr the fraction of document entities reached by any query entity. We fuse with the dense score,

$$s = (1 - \alpha) s_{\text{dense}} + \alpha s_{\text{kg}}, \quad (1)$$

with α the *KG weight* and $s_{\text{dense}} = \sigma / \max_q \sigma$ a per-query max-only rescaling of the raw Contriever score σ (the Appendix explains why not min-max). We sweep $k \in \{1, 2, 3\}$, $t \in \{500, 1k, 2k, 5k, 10k, \infty\}$, $\alpha \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$: 90 rerank cells plus the dense baseline.

4. Results

Baseline is sound. Dense retrieval scores 0.364, in line with the ≈ 0.36 that Cuconasu et al. [3] report for Llama-2 with Contriever-retrieved documents on NQ-open — the harness is healthy.

No gain, then harm. The best cell ($\alpha=0.3$, dist 2, $t=5k$) reaches 0.372: $+0.8\text{pp} \approx 8$ queries, i.e. noise (full grid in Table 1, Appendix). As α grows the KG signal dominates and accuracy falls monotonically, bottoming at 0.290 ($\alpha=0.9$, dist 3, $t=5k$).

It reshuffles, but does not help. The rerank is far from inert: at $\alpha=0.5$ it rewrites the top-5 for 84–99.5% of queries (mean Jaccard@5 0.11–0.30 — typically barely one of five passages survives), yet accuracy stays at the baseline.

Threshold gradient is the mechanism. Holding $\alpha=0.9$, dist 2, accuracy decays monotonically as the hub filter loosens, from 0.354 ($t=500$) to 0.295 ($t=\infty$): admitting high-degree nodes as bridges connects everything to everything and destroys the signal.

Significance. Exact McNemar tests against the baseline, Bonferroni-corrected for the 90 comparisons (significance threshold $0.05/90 \approx 5.6 \times 10^{-4}$), flag 13 of the 90 conditions as significant — *all* worse, none better; each flips 170–224 of the 1000 answers yet nets negative.

5. Discussion and conclusions

Why it fails. On a graph as dense as Wikidata, 2–3-hop reachability *saturates*: almost every document entity is reachable from almost every query entity, so $\text{cr} \approx 1$, the score collapses onto pr alone, and discrimination is lost; the hub filter limits the damage but recovers no signal. For about 17% of queries the question entity is itself a very high-degree node (a country, a religion), where any passage is graph-close and the KG score is uninformative.

Limitations and future work. Our negative result is scoped to one operationalisation of topology: binary, unweighted set-overlap, fused linearly with a max-only-scaled dense score. Two variants remain untested: weighting bridges by inverse degree (which directly targets the hub mechanism we diagnosed) and rank-based fusion (which removes the scale mismatch between the two scores). The deeper limitation, however, is that the metric collapses relation semantics (P31 *instance-of* and P19 *place-of-birth* count alike), so it cannot screen out semantically irrelevant links: more than any reweighting, it is the graph’s relations and semantics that the next iteration should exploit.

Conclusion. Pure topological proximity does not filter distractors for dense RAG on NQ-open; the value is the explained negative result and the reusable Wikidata tooling.

Data and code

- {1} Corpus: https://huggingface.co/datasets/florin-hf/wiki_dump2018_nq_open.
- {2} Queries: https://huggingface.co/datasets/florin-hf/nq_open_gold.
- {3} Wikidata HDT dump: <https://hdt-dumps.cluster.ai.wu.ac.at/dumps/>.
- {4} Code: <https://github.com/Gian-ream/dl-RAG-denseAndKG>.
- {5} Data: <https://drive.google.com/drive/folders/1RlQuowaWmVF0fe2fcAqVBBaT4dmsnU7s>.

Statement on the use of AI

This project was developed in close collaboration with Claude (Anthropic). Claude wrote most of the notebook and script code under the author’s step-by-step direction and review (explain-then-approve), patched the environment issues documented in the Appendix, suggested literature, kept the project log of every design decision and change of direction, helped polish the original proposal, and drafted the LaTeX of this report, then reshaped by the author. Key ideas came from both sides: the boolean-reachability reformulation and the max-only argument are the author’s, their implementation Claude’s. Hypothesis, methodological decisions, experimental runs are the author’s; every reported number was re-verified against the raw outputs, with corrections

flowing in both directions.

References

- [1] Ayoola, T., Tyagi, S., Fisher, J., Christodoulopoulos, C., and Pierleoni, A. ReFinED: An efficient zero-shot-capable approach to end-to-end entity linking, 2022. URL <https://arxiv.org/abs/2207.04108>.
- [2] Bast, H., Hertel, M., and Prange, N. A fair and in-depth evaluation of existing end-to-end entity linking systems, 2023. URL <https://arxiv.org/abs/2305.14937>.
- [3] Cuconasu, F., Trappolini, G., Siciliano, F., Filice, S., Campagnano, C., Maarek, Y., Tonellotto, N., and Silvestri, F. The power of noise: Redefining retrieval for RAG systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR 2024, pp. 719–729. ACM, 2024. doi: 10.1145/3626772.3657834. URL <http://dx.doi.org/10.1145/3626772.3657834>.
- [4] Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., and Mian, A. A comprehensive overview of large language models, 2024. URL <https://arxiv.org/abs/2307.06435>.
- [5] Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., and Tang, S. Graph retrieval-augmented generation: A survey, 2024. URL <https://arxiv.org/abs/2408.08921>.

Appendix

Accuracy of all rerank settings

Table 1 reports the LLM-judge accuracy of every rerank setting in the sweep — all 90 (k, t, α) cells — against the dense baseline discussed in the Results section.

Pipeline: notebook flow

The project is organised as nine notebooks, numbered in execution order. Each notebook reads the artifacts written by the previous ones under `data/` and writes its own, so every stage is independently resumable and inspectable. The Wikidata graph itself is built separately by the scripts in `scripts/pipeline/`, which produce the edge list, the per-node degree statistics and the precomputed 1-hop neighbourhoods that notebook 07 consumes; the `pyHDT`-based ones run under WSL2, the rest natively on Windows (see Environment).

Table 1. LLM-judge accuracy, all 90 rerank settings (1000 queries); dense baseline = 0.364; $t=\infty$ means no hub filter. Distance 1 has no bridge node, hence threshold-invariant. Best **bold**, worst underlined.

k	t	$\alpha=.1$	.3	.5	.7	.9
1	any	.353	.344	.343	.316	.322
2	500	.362	.358	.361	.348	.354
2	1,000	.364	.361	.358	.343	.350
2	2,000	.360	.371	.357	.337	.341
2	5,000	.357	.372	.362	.337	.337
2	10,000	.364	.365	.357	.334	.329
2	∞	.354	.343	.311	.298	.295
3	500	.354	.344	.341	.329	.335
3	1,000	.346	.355	.342	.322	.310
3	2,000	.354	.349	.316	.303	.293
3	5,000	.366	.351	.310	.292	<u>.290</u>
3	10,000	.361	.362	.321	.306	.304
3	∞	.347	.333	.314	.314	.318

1. `01_corpus_preparation` — downloads the corpus and segments every article into sentence-aligned, 100-word passages, producing the $\sim 42\text{M}$ -passage retrieval corpus.
2. `02_nq_filtering` — filters NQ-open down to the queries whose answers are short (≤ 5 tokens) and whose question and answers all carry ReFinED-linkable entities.
3. `03_embedding` — encodes every passage with Contriever and builds nine FAISS shards (the full index does not fit in memory as a single block).
4. `04_answer_preparation` — retrieves the top-100 passages for each query of the 1000-query subset and entity-links every retrieved passage.
5. `05_answer_curation` — identifies the 344 queries whose entire top-100 carries no linkable entities and selects replacement queries from a candidate pool.
6. `06_apply_curation` — applies the substitution, producing the `*_curated` files used by everything downstream.
7. `07_kg_rerank` — computes KG reachability (cr/pr) for every query–passage pair over the full $k \times t$ grid, plus the Jaccard reshuffle diagnostics.
8. `08_llm_eval` — has Llama-2-7B generate an answer for each query under all 91 conditions (one baseline + 90 rerank cells).
9. `09_llm_judge` — has Qwen2.5 judge every answer against the NQ gold answers and aggregates accuracy and the McNemar tests with Bonferroni correction.

A note on reproducibility: to work with the pipeline’s data directly, download the full `data/` folder from {5} ($\sim 614\text{ GB}$ in total; it also includes the smaller artifacts already versioned in the code repository). Alternatively, ev-

ery artifact except the HDT dump and its index — which must be fetched manually from `{3}` — can be re-created by the notebooks themselves, which download their own sources on first run. We advise against this route, however: the overall computational cost is high and, depending on the stage and the machine, a full rerun of a single notebook can take several days. One exception: notebook 09 is re-runnable out of the box — its inputs are versioned in the code repository and the judge model is downloaded automatically — although it performs 91,000 Qwen judgments, so a full pass still takes time.

Environment

Exact reproduction requires *two* environments, because no single platform runs the whole pipeline. The requirements are:

- **Windows + uv.** The main environment: every notebook runs here, with dependencies pinned in `pyproject.toml` and installed via `uv sync`.
- **WSL2/Ubuntu for the graph build.** `pyHDT` requires a C++ build that does not compile cleanly on native Windows, so the three HDT scripts (edge export, label lookup, completeness check) run in a Linux environment that reads the 166 GB dump through `/mnt/c`. The pre-built index file (`.hdt.index.v1-1`, 107 GB, from the same source) must sit next to the dump: without it `pyHDT` attempts to build the index in RAM, which exceeds WSL2’s memory cap.
- **faiss-gpu via conda.** There is no Windows pip wheel for the GPU build of FAISS, so it lives in a separate miniconda installation and is bridged into the `uv venv` at runtime (`add_dll_directory` for the DLLs plus a `sys.path` insert for the package); the conda paths are machine-specific and must be adapted in notebook 04.
- **ReFinED patch.** ReFinED V1 breaks on Windows + Python 3.12 + transformers 4.48 through three distinct bugs: a Unix-only `strftime("%s")` call in its model downloader, `add_special_tokens` passed as a tokenizer-loading kwarg (an error on recent transformers), and regex patterns written without raw strings (a Python 3.12 warning). `scripts/tooling/patch.refined.py` patches the installed package in place; it is idempotent and the notebooks that load the model invoke it automatically at every run, so even a fresh reinstall is re-patched on the next execution.
- **HuggingFace token.** A personal token must be placed in the `.hf.token` file at the repo root (gitignored). It is strictly required only for the licence-gated Llama-2; the other notebooks also read it, but merely

for faster downloads (Qwen2.5 is ungated).

The schema of every data artifact is catalogued in `documents/DATA_DICTIONARY.md`; the artifacts themselves can be downloaded from `{5}`.

Design choices

Dense normalization: max-only, not min-max. The raw Contriever score σ is an inner product over embeddings that are not L2-normalised, so it is not bounded by 1 (within a query, top-100 values can exceed it) and sits in a narrow per-query band. Before fusing it with the KG score we rescale it by the per-query maximum only, $s_{\text{dense}} = \sigma / \max_q \sigma$, which caps it at 1 — as the fusion in Eq. (1) assumes — without touching the relative gaps. The obvious alternative, min-max normalisation, would stretch the narrow band to the full $[0, 1]$ interval, and in doing so it would change the *philosophy* of the ranking, not just its scale. A worked example: passages A($\sigma=1.05$, $s_{\text{kg}}=0.2$) and B($\sigma=1.00$, $s_{\text{kg}}=0.4$), fused at $\alpha=0.5$. Max-only divides by 1.05, so $s_{\text{dense}}^A = 1.0$ and $s_{\text{dense}}^B = 0.95$, and the fused scores are $s^A = 0.5 \cdot 1.0 + 0.5 \cdot 0.2 = 0.60$ versus $s^B = 0.5 \cdot 0.95 + 0.5 \cdot 0.4 = 0.68$: **B wins**. Min-max instead maps the two dense scores to the extremes, $s_{\text{dense}}^A = 1$ and $s_{\text{dense}}^B = 0$, giving $s^A = 0.60$ versus $s^B = 0.5 \cdot 0 + 0.5 \cdot 0.4 = 0.20$: the ranking flips to **A** on the strength of a dense difference that is in fact negligible.

Reader prompt. We use Llama-2-7B *base*, not the chat variant, so the model is a pure next-token completer with no instruction tuning: prompted naively, it continues “Answer:” with Wikipedia-style prose rather than a short extracted span. We therefore prepend a one-shot extraction example that demonstrates both the task and the output format. We also place the question *above* the documents, where Cuconasu et al. [3] place it after them: the model reads the question first and can attend back to it while scanning the passages. Following the paper’s own finding on position bias, the top-ranked passage is placed last in the context, immediately above the “Answer:” slot where the model attends most. The full template:

```
You are given a question and you
MUST respond by EXTRACTING the
answer (max 5 tokens) from one of
the provided documents. If none of
the documents contain the answer,
respond with NO-RES.

Example:
Question: who wrote hamlet
Document [1] (Title: Macbeth) ...
...documents [2]-[4]...
Document [5] (Title: Hamlet)
...written by William Shakespeare...
Answer: William Shakespeare
```

```

Question: <question>
Document [1] (Title: ...) <rank-5
passage>
...
Document [5] (Title: ...) <rank-1
passage>
Answer:
    
```

Judge. Substring exact-match — the metric used by Cucunaru et al. [3] — misses answers that are correct but paraphrased, reformatted, or wrapped in a preamble. We therefore score with an LLM judge that answers YES/NO to “does the response contain one of the gold answers?”. The judge must be a *different* model from the generator to avoid self-preference bias, so we use Qwen2.5-7B-Instruct (Apache-2.0, ungated): strong enough to absorb paraphrase and formatting, cheap enough to judge all 91,000 responses locally. We manually spot-checked random samples of TRUE and FALSE verdicts (notebook 09) to confirm the judge behaves as intended. The judge prompt:

```

system: You are a strict judge.
Given a question, a list of
acceptable answers, and a model
response, decide whether the
response contains at least one of
the acceptable answers (verbatim or
as a clear paraphrase). Reply with
exactly YES or NO, nothing else.

user: Question: <question>
Acceptable answers: <gold answers>
Response: <Llama-2 response>

Does the response contain at least
one of the acceptable answers?
Reply YES or NO.
    
```

What we tried first

Several pieces of the final pipeline replaced a first attempt that failed for an instructive reason.

Query curation. Once retrieval was in place we found that 344 of the 1000 subset queries had a top-100 with no linkable entities at all, leaving the KG score undefined exactly where it was supposed to act. We replaced those queries with entity-bearing ones drawn from a larger candidate pool. Note the direction of the bias this introduces: the evaluation set is tilted *in favour* of the KG signal, so the negative result should be read as “even on KG-favourable queries, topological proximity does not help”.

Local triple store → SPARQL → local HDT. Our first attempt at graph access was to download Wikidata and host it locally in a Java triple store (Blazegraph), so we could navigate it without network limits. Ingestion never completed: the working space needed to build the indices blew past the 5 TB of disk we had available. We fell back to the public Wikidata SPARQL endpoint, which collapsed immediately on popular entities: even a minimal COUNT over

a single hub (?x wdt:P31 wd:Q5, “instances of *human*”) exceeded the server’s limit, and with it fell the whole fallback scheme we had designed around it. We finally switched to a pre-built HDT dump of Wikidata queried locally with pyHDT, where the same predicate-restricted counts go straight to a precomputed index and return in well under a millisecond, with no rate limits.

Wildcard → per-predicate export. Our first edge exporter walked the whole dump with a wildcard triple iterator and filtered entity-to-entity statements in the loop. On a dump this size the iterator misbehaved in two ways at once: it *over*-iterated (yielding far more triples than the file header declares) while genuine Q-Q edges were still *missing* from its output — an inflated traversal with missing data and no error raised. Restricting the iteration to one predicate at a time turned out to be exact: per-predicate counts match the HDT header, the run is deterministic and verifiable, and it yielded ~471k more edges than the aborted wildcard run had written, for a final count of 661,471,158.

BFS-N3 → meet-in-the-middle. The proposal’s natural reading — precompute the full 3-hop neighbourhood of every seed by BFS — timed out on 89% of seeds in a dry run, including perfectly ordinary, low-degree entities. The cause is the cumulative wave explosion — even when the seed is small, the second wave reaches tens of thousands of nodes and the third has to iterate millions of triples — aggravated by the fact that our reachability is direction-agnostic, so the BFS walks the directed graph *ignoring edge direction* and every wave can also flow backwards into popular targets. The fix came from noticing that the metric never needs the full neighbourhood, only the *boolean* question “is this document entity within 3 hops of this query entity?”. That question can be answered by precomputing 1-hop neighbourhoods alone (~93.5M rows) and meeting in the middle: distance 1 is a set lookup, distance 2 a set intersection, distance 3 a single indexed edge-probe between the two 1-hop sets. A hub’s neighbour list is never enumerated — we only test membership in it.